

الحالة الحالية — VIXOR MASTER V2

تدقيق

تقييم شامل لجاهزية التطبيق ونقاط الفشل

VIXOR MASTER V2 — Current State Audit

Audit date: 2025-06-17 **Auditor:** Explore sub-agent (single-pass deep audit) **Scope:** `/home/z/my-project/src/` + root config + migrations **Method:** Read every file referenced in the task brief; cross-checked against `agent-ctx/*.md`, `worklog.md`, `plan.md`.

1. Executive Summary — Health Rating: 4 / 10

The codebase is **architecturally sophisticated but operationally crippled by missing environment configuration and several latent defects that the previous agents either caused or failed to surface**. On paper, VIXOR V2 implements a multi-domain DDD-style architecture with a local SMC/ICT analysis engine, multi-agent copilot, debate engine, risk governor, paper trading, chart-truth layer, and P1 intelligence layer (tools + memory + events). In practice, the local sandbox at `/home/z/my-project/` cannot run end-to-end because only one of ~13 required env vars (`DATABASE_URL`) is set.

User-reported problem	Root cause found in audit
Top dimensions/layout misaligned	<code>AppShell</code> uses <code>max-w-md lg:max-w-4xl</code> for header, main, and bottom-nav — narrow <code>max-w-md</code> (448px) on mobile creates "floating" centered bottom nav; on desktop <code>max-w-4xl</code> (896px) is too narrow for a "Bloomberg Professional" trading app. Header is <code>fixed top-0</code> but main has <code>pt-20</code> which is correct — but the header itself has safe-area padding that can push it past 56px on iOS PWA. Bottom nav uses <code>pb-3</code> outside the safe-area calc so it can sit too high on iOS.
Pages not smoothly connected	<code>route.tsx</code> auth-guard catches errors and silently redirects to <code>/auth</code> (line 19-23) — every server-fn failure during navigation looks like a logout. Plus, <code>useStableServerFn</code> + <code>useRenderGuard</code> exists to fight React #310 loops that prior agents hit. SSR is disabled on <code>_authenticated</code> (<code>ssr: false</code>) so first paint is blank ▯ spinner ▯ page, which feels "disconnected".
Data display not smooth/professional	Most data fetches silently return <code>[]</code> or <code>null</code> when env vars are missing (price-fetcher returns null ▯ <code>fetchPrices</code> returns <code>[]</code> ▯ dashboard shows empty "Market data temporarily unavailable" card). 78 <code>console.warn</code> calls + 52 <code>console.error</code> calls — failures are swallowed rather than surfaced to the user.
UI/UX not professional	Mixed design language: appshell claims "Bloomberg terminal" but uses emoji (▯ ▯ ▯) in pair lists; lots of <code>text-[9px]</code> / <code>text-[10px]</code> uppercase tracking-widest labels creating a "Bloomberg-try-hard" aesthetic that doesn't mesh with rounded-2xl <code>gradient-primary</code> buttons.

User-reported problem	Root cause found in audit
Some features/pages don't work	All server fns behind <code>requireSupabaseAuth</code> middleware throw "Missing Supabase environment variable(s): SUPABASE_URL" because env not set. The deep-no-op Proxy on the browser client (<code>supabase/client.ts</code>) returns <code>{data:null, error:"Supabase not configured"}</code> for every call, so every list page shows empty state.
Telegram login doesn't work	<code>TELEGRAM_BOT_TOKEN</code> not set in <code>.env</code> → <code>telegramSignIn</code> server fn throws on line 24 of <code>auth.functions.ts</code> . Also <code>VITE_TELEGRAM_BOT_USERNAME</code> not set → falls back to "VixorAIBot" hard-coded. Login Widget requires BotFather <code>/setdomain</code> to be configured for the deployment URL — a manual step the user must perform.
Chart analysis returns "Unable to identify asset"	This string is dead code in the live pipeline — <code>formatExtractionFailureMessage</code> in <code>chart-context.ts:145</code> is never called . <code>chart-validation.ts</code> always returns <code>valid: true</code> (line 41 comment: "ALWAYS valid — never block analysis"). The error only surfaces if a user views an old analysis row that was created before the fix and stored <code>error_message</code> containing that string. The current code in <code>analysis/functions.ts:285-291</code> explicitly rewrites this message to a friendlier one. Verdict: this is not a live defect — the user is seeing stale DB rows from old failed runs.

2. Tech Stack & Config

Stack (from `package.json`)

- **Framework:** TanStack Start `1.168.25` + TanStack Router `1.170.15` + React `19.2.0` + Vite `7.3.1` + Nitro `3.0.260603-beta`
 - **Deploy target:** Vercel preset (`vite.config.ts:24`)
 - **DB:** Supabase JS SDK `2.107.0` + pg `8.21.0`
 - **Cache:** Upstash Redis `1.38.0` (with in-memory fallback)
 - **UI:** TailwindCSS `4.2.1` (Vite plugin) + 30+ Radix UI primitives + shadcn/ui "new-york" style + `lucide-react` + `framer-motion` + `recharts` + `lightweight-charts 5.2.0`
 - **AI:** `z-ai-web-dev-sdk 0.0.18` (VLM via `glm-4.6v`) + Vercel AI SDK `6.0.197` with `@ai-sdk/google` and `@ai-sdk/openai-compatible`
 - **Forms/validation:** `react-hook-form` + `zod 4.4.3`
 - **Telegram:** `@telegram-apps/sdk 3.11.8` + custom HMAC verification
- **Other:** `docx` (export), `html2canvas` (chart screenshots), `date-fns`, `sonner` (toasts), `vaul` (drawer)

Scripts

dev → vite dev (port 8080, host "::")

```

build    → vite build && node scripts/fix-vercel-bundle.mjs
lint     → eslint .
format   → prettier --write .

```

Critical config observations

- **vercel.json** has only ONE cron entry: `0 0 * * * /api/generate-signals`. Missing: `/api/check-alerts` (also exists, runs alert checks). Alerts will not trigger automatically.
- **vite.config.ts** registers 7 Nitro API handlers explicitly (lines 31-39). All 7 must exist at those exact paths or build fails.
- **tsconfig.json** is strict (`"strict": true`) but disables `noUnusedLocals` and `noUnusedParameters` — dead-code-friendly but allows unused imports to accumulate.
- **components.json** has `"rtl": false` — yet `src/shared/i18n/` implements Arabic RTL. The shadcn generator won't add RTL-aware variants to new components.
- **bunfig.toml** enforces 24h supply-chain guard (good security practice).

Environment variables (status in `/home/z/my-project/.env`)

Variable	Required by	Status
DATABASE_URL	(referenced in plan.md, not in code)	❑ SET (only one)
SUPABASE_URL	All supabase clients	❑ MISSING
SUPABASE_SERVICE_ROLE_KEY	<code>client.server.ts</code> admin client	❑ MISSING
SUPABASE_ANON_KEY / SUPABASE_PUBLISHABLE_KEY	<code>client.ts</code> browser, <code>auth-middleware.ts</code>	❑ MISSING
VITE_SUPABASE_URL / VITE_SUPABASE_PUBLISHABLE_KEY	Browser bundle	❑ MISSING
TELEGRAM_BOT_TOKEN	<code>auth.functions.ts:24</code> , <code>linkTelegramAccount</code> , <code>alert-checker</code> , <code>telegram-webhook</code>	❑ MISSING
VITE_TELEGRAM_BOT_USERNAME	<code>auth.tsx:42</code> (defaults to "VixorAIBot")	❑ MISSING
TELEGRAM_WEBHOOK_SECRET	<code>routes/api/-telegram-webhook.ts:13</code>	❑ MISSING
TWELVEDATA_API_KEY	<code>price-fetcher.ts</code> (4 sites), <code>twelvedata.ts:24</code>	❑ MISSING
FINNHUB_API_KEY	<code>market/functions.ts:31</code> , <code>economic-calendar.ts:144</code>	❑ MISSING
UPSTASH_REDIS_REST_URL + UPSTASH_REDIS_REST_TOKEN	<code>shared/cache.ts:238-239</code> (falls back to in-memory)	❑ MISSING (optional, fallback works)
CRON_SECRET	All <code>/api/*</code> routes for non-Vercel-Cron callers	❑ MISSING
ENABLE_PAPER_TRADING	<code>paper.engine.ts:76</code> (gates paper trading)	❑ MISSING (paper trading disabled)
ENABLE_DEBATE_ENGINE	<code>run-analysis.ts:307</code> (gates debate engine)	❑ MISSING (debate engine disabled)

Variable	Required by	Status
<code>NODE_ENV</code>	Several code paths	Set automatically by Vercel/Vite

Note: The `worklog.md` (Task 3) claims all these vars ARE set on Vercel production (`vixor-app.vercel.app`). So the user's complaints likely come from running the app locally against the sandbox (where only `DATABASE_URL` exists), OR from production where BotFather `/setdomain` was never run.

3. App Shell & Layout — Specific Bugs

`src/routes/__root.tsx` (321 lines)

- RootShell** sets `<html lang="en" className="dark">` — but `applyDirection()` in `il8n/index.tsx:57` sets `lang="ar"` and adds `.rtl` class only after mount. Causes a brief LTR-then-RTL flash on Arabic users.
- Theme bootstrap** is duplicated: defined both in `head.scripts[]` (lines 158-161) AND in inline JSX `<script dangerouslySetInnerHTML>` (lines 182-186). Both write the same IIFE. The first one is reported in `worklog.md` as "silently dropped by TanStack Start's `scripts[]` config" — so only the JSX one actually fires. Dead config in `scripts[]` array.
- Auth state subscription** is attached via dynamic `import("@/shared/supabase/client")` and stored on `window.__vxAuthSub` (line 290) — never unsubscribed on unmount (the cleanup only clears `authDebounce`, not the subscription). Memory leak across HMR reloads in dev.
- `useNavigate()` ref pattern** — comment claims it fixes React #310 but it's a workaround; the actual fix is in `useStableServerFn` which avoids `useRouter()`.

`src/components/vixor/AppShell.tsx` (261 lines) — Layout bugs

Bug	Location	Severity
<code>max-w-md lg:max-w-4xl</code> constraint on mobile bottom-nav	line 200	HIGH — on devices >448px, the bottom nav floats centered with empty space on both sides (looks disconnected from screen edges)
Header also constrained to <code>max-w-md</code> on mobile	line 124	HIGH — same floating effect; header doesn't span full width on phablets
Header <code>h-14</code> (3.5rem) + <code>pt-20</code> (5rem) on main = 1.5rem breathing room	lines 99, 125	MEDIUM — too much top whitespace below header
Bottom nav: <code>pb-3 + env(safe-area-inset-bottom)</code> on parent	lines 197-200	MEDIUM — double padding on iOS PWA, bottom nav floats higher than expected
<code>signedIn = path !== "/auth"</code>	line 51	LOW — pure client-side check, won't catch the auth state during SSR transition

Bug	Location	Severity
Telegram auto-link runs on every signedIn mount	lines 64-81	LOW — wastes a server roundtrip if already linked (only localStorage guards it)
Hardcoded "ME" initials in profile avatar	line 171	LOW — never shows user's actual photo/initials
lg:max-w-4xl is 896px — too narrow for "Bloomberg Professional" desktop trading app	line 99	MEDIUM — leaves 60%+ of horizontal screen unused on 1440p displays
Tab "Portfolio" matches 10 different paths	lines 31-42	LOW — settings/premium/referral all highlight Portfolio tab; user has no idea where they are
Bottom-nav uses pointer-events-none on container, pointer-events-auto on inner	lines 197, 200	OK — intentional pattern to allow tap-through above the nav

src/components/vixor/atoms.tsx (442 lines)

- highlightSMCTerms (line 348) runs text.split(regex) on every render — expensive for long analysis text. No memoization.
- EducationLayer rebuilds Object.keys(SMC_GLOSSARY) array on every render (line 374). Should be module-level constant.
 - ConfidenceBar has no aria attributes — accessibility issue.
- RecBadge uses text-[11px] (sm) and text-sm (lg) — inconsistent with rest of app which uses text-xs/text-sm.

RTL handling

- i18n/index.tsx correctly toggles dir and .rtl class.
- styles.css has .rtl overrides for terminal-card-accent, scenario-primary/alternative/counter, bg-gradient-to-r, text-left/right, items-start (lines 385-427).
- BUT: ExpandableWidget uses borderInlineStartColor (logical property — good, RTL-safe) while atoms.tsx uses physical border-left-style classes in some places. Inconsistent.

Mobile responsiveness gaps

- No tablet breakpoint (only lg: at 1024px). Between 640-1023px everything renders as mobile.
- grid-cols-2 lg:grid-cols-4 in dashboard Market Pulse — on tablet (768-1023px) shows 2 cols which is fine but wastes space.
- copilot.tsx (1439 lines) — uses flex flex-col h-[calc(100vh-...)] patterns that break on landscape mobile.

4. Routes Inventory

All 17 routes + auth + root route. All `_authenticated/*` routes have `ssr: false` (set in `route.tsx`) — first paint is always a blank shell.

#	Route file	Path	Works?	Issues
1	<code>__root.tsx</code>	(shell)	☐☐	Duplicated theme script, auth-sub memory leak, theme/RTL flash
2	<code>auth.tsx</code>	<code>/auth</code>	☐☐	Login Widget requires BotFather <code>/setdomain</code> (manual user step); <code>TELEGRAM_BOT_TOKEN</code> env required for WebApp flow; falls back to email/password which requires Supabase Auth enabled
3	<code>_authenticated/route.tsx</code>	(guard)	☐☐	Catches ALL errors and redirects to <code>/auth</code> — masks server-fn failures as "logged out"
4	<code>_authenticated/index.tsx</code>	<code>/</code>	☐☐	950 lines. 7 <code>useQuery</code> calls. All queries fail silently when env missing. Empty states everywhere. <code>lg:grid lg:grid-cols-2</code> pattern at line 494 produces <code>space-y-5 lg:space-y-0</code> switching that causes layout reflow on resize.
5	<code>_authenticated/signals.tsx</code>	<code>/signals</code>	☐☐	547 lines. Strategy editor + paginated signals list. Pagination works server-side but filters are client-side only — page reset doesn't preserve filter.
6	<code>_authenticated/portfolio.tsx</code>	<code>/portfolio</code>	☐☐	1218 lines! Trades CRUD + stats + equity curve (recharts). <code>"use client"</code> directive at top — redundant in TanStack Start. Equity curve uses 200 points hard-coded.
7	<code>_authenticated/journal.tsx</code>	<code>/journal</code>	☐☐	615 lines. Tabs: overview/history/notes/reports. Reports tab is mostly empty placeholder.

#	Route file	Path	Works?	Issues
8	<code>_authenticated/trade-desk.tsx</code>	<code>/trade-desk</code>	☐☐	406 lines. Lot calculator + open positions. Only 5 pairs supported (<code>XAUUSD</code> , <code>EURUSD</code> , <code>GBPUSD</code> , <code>USDJPY</code> , <code>BTCUSD</code>) — out of sync with asset registry which has 20+ pairs.
9	<code>_authenticated/copilot.tsx</code>	<code>/copilot</code>	☐☐	1440 lines! Multi-agent chat with consensus mode. Layout uses absolute positioning that breaks on small screens. Conversations list panel toggles with <code>PanelLeftClose/PanelLeftOpen</code> but no persistence.
10	<code>_authenticated/daily-loop.tsx</code>	<code>/daily-loop</code>	☐☐	1151 lines. 3-phase daily workflow (morning/session/EOD). Uses <code>updateStreak</code> server-side which has a bug: if <code>existing.last_completed_date</code> is null, <code>lastDate</code> is null, <code>lastStr</code> is null, comparison <code>lastStr === yesterdayStr</code> is false, streak resets to 1.
11	<code>_authenticated/analyze.tsx</code>	<code>/analyze</code>	☐☐	430 lines. Image upload + pair selection. <code>pickFile()</code> infers pair from filename (line 142-145) — bad UX, should use VLM detection. URL screenshot parameter passed via <code>?screenshot=base64...</code> — huge URL, hits browser limits.
12	<code>_authenticated/analysis.\$id.tsx</code>	<code>/analysis/\$id</code>	☐☐	948 lines. Polls <code>getAnalysis</code> every 3s. If <code>status=="failed"</code> , shows <code>a.error_message</code> — this is where the stale "Unable to identify asset" string surfaces to users.

#	Route file	Path	Works?	Issues
13	<code>_authenticated/charts.tsx</code>	<code>/charts</code>	☐☐	496 lines. TradingView widget integration. "Analyze" button has 60-second cooldown. OHLCV fetch uses <code>INTERVAL_MAP</code> reverse lookup that may fail for non-standard intervals.
14	<code>_authenticated/discover.tsx</code>	<code>/discover</code>	☐☐	1307 lines! 5 tabs: watchlist/scanner/calendar/news/heatmap. Heatmap tab has "coming soon" note for interactive features — partial implementation.
15	<code>_authenticated/notifications.tsx</code>	<code>/notifications</code>	☐☐	99 lines. Minimal. No pagination, hardcoded 50-item limit in server fn. No notification preferences UI.
16	<code>_authenticated/premium.tsx</code>	<code>/premium</code>	☐☐	215 lines. Plans + packs + Telegram Stars payment. <code>subscribePremium</code> server fn just inserts a row with <code>current_period_end = now + 30/365 days</code> — no actual payment verification (worklog calls this "instant grant").
17	<code>_authenticated/referral.tsx</code>	<code>/referral</code>	☐☐	170 lines. 4-tier system (Bronze/Silver/Gold/Diamond). <code>claimReferral</code> server fn credits both parties — but if <code>credit_points</code> RPC fails, code throws and rollbacks nothing.
18	<code>_authenticated/profile.tsx</code>	<code>/profile</code>	☐☐	318 lines. Badges are hardcoded (line 30) — no actual achievement logic. XP bar uses <code>Math.floor(xp / 100) + 1</code> — every 100 XP = 1 level, but no level-up notification.

#	Route file	Path	Works?	Issues
19	<code>_authenticated/settings.tsx</code>	<code>/settings</code>	☐☐	337 lines. Theme toggle, language picker, sign-out. Haptics/sound/price-alerts/news-alerts toggles are non-functional — they save to local state but no effect. No "Account" section to manage Telegram link.
20	<code>_authenticated/referral.tsx</code>	(dup)	—	(already listed)

API routes (`src/routes/api/`)

- `-check-alerts.ts` (leading-dash = ignored by router, manual handler) + `check-alerts.ts` — DUPLICATE. Same for `-generate-signals.ts/generate-signals.ts`, `-telegram-webhook.ts/telegram-webhook.ts`, `-migrate.ts/migrate.ts`. The dash-prefixed versions are the ones actually wired in `vite.config.ts:32-39`. The non-dashed versions are dead code that the router tries to mount as routes.
- All API routes gate on `CRON_SECRET` for non-Vercel-Cron callers. In production with no `CRON_SECRET`, they return 500 "Cron not configured".

5. Domain Modules Map

```
src/domains/
├─ analysis/           ← Local SMC/ICT engine (DETAILED in §6)
├─ chart-intelligence/ ← VLM extraction (DETAILED in §7)
├─ chart-truth/        ← P1.6 truth layer – validates vision price vs market price (NEVER blocks)
├─ copilot/            ← Multi-agent chat (4 agents: market_analyst, risk_manager, news_analyst, strategy_build
├─ daily-loop/         ← 3-phase daily workflow
├─ debate/             ← 4-agent debate engine (analyst/strategist/risk-guard/contrarian) – GATED by ENABLE_DEB
├─ market/             ← price-fetcher, economic-calendar, twelvedata (DETAILED in §6)
├─ notes/              ← trading_notes CRUD
├─ paper-trading/      ← Paper engine + trade ledger – GATED by ENABLE_PAPER_TRADING
├─ risk-governor/      ← Synchronous risk evaluator (PROCEED/REDUCE_SIZE/WAIT/BLOCK)
├─ trades/             ← Real trades CRUD + stats + equity curve
├─ trading/            ← Price alerts + daily signals + user strategies
├─ user/               ← Auth, profile, commerce, notifications, referrals, Telegram link
└─ watchlist/         ← Watchlists CRUD
```

Cross-cutting shared infrastructure (`src/shared/`)

- `asset-registry/` — Single source of truth for ~20 trading pairs with Binance/TwelveData/TradingView/Finnhub symbol mappings + per-pair config (decimals, volatility, basePrice).
- `cache.ts` — Hybrid cache (Upstash Redis primary, in-memory fallback). All API calls cached here.
- `events/` — Typed event bus with Supabase `domain_events` persistence. Non-blocking.

- **memory/** — **user_memories** table (5 categories: preference/behavior/mistake/insight/strategy). Used by copilot agent.
 - **supabase/** — Lazy-initialized clients with deep-no-op Proxy fallback (see §9).
- **tool-registry/** + **tool-router/** — Tool dispatch layer for copilot agent. Currently registers 2 tools (**createAlert**, **journal-analysis**).
 - **i18n/** — English + Arabic translations, RTL handling.

Duplicate **src/lib/** folder

After the architecture refactoring (agent-ctx/6), **src/lib/** was converted to backward-compat re-export barrels. Most files are 1-3 line re-exports. **src/lib/vixor.functions.ts** is still actively imported by **every route** — it's the master barrel re-exporting from **@domains/*/functions**. **This is intentional and works**, but creates an extra layer of indirection.

Duplicate **src/integrations/supabase/** folder

Also backward-compat re-exports to **@shared/supabase/**. Same pattern.

Duplicate **src/server/*.server.ts** files

All are 1-3 line re-exports (e.g. **src/server/run-analysis.server.ts** → **@domains/analysis/server/run-analysis**). The **src/server/api/** folder doesn't exist — the actual API handlers are in **src/routes/api/**. The **root-level /server/api/ folder** is the one wired into **vite.config.ts** Nitro handlers.

6. Analysis Engine Deep Dive — Is it actually running?

Architecture

User uploads image

```

↓
createAnalysis (functions.ts:28)
  ↓ requires Supabase auth
  ↓ inserts analyses row (status="processing")
  ↓ fetches real OHLCV from Binance → TwelveData (4 fallback sources)
  ↓ calls runChartAnalysis (run-analysis.ts:173)
    ↓ Step 1: extractChartContext (z-ai VLM glm-4.6v)
    ↓ Step 2: validateChartContext (SOFT — never blocks, warnings only)
    ↓ Step 2.5: validateChartTruth (compare vision price vs real market)
    ↓ Step 3: determine pair (vision > user-selected > filename > "EUR/USD")
    ↓ Step 4: runLocalAnalysis (engine.ts:68) — THE CORE ENGINE
      ↓ generate real or synthetic OHLCV bars
      ↓ analyzeMarketStructure (swing points, BOS, CHoCH)
      ↓ detectOrderBlocks, detectFVGs, detectLiquidityZones, detectSRLevels
      ↓ detectCandlestickPatterns (74 patterns), detectChartFormations (20), detectHarmonicPatterns (8)
      ↓ getLatestIndicators (RSI, MACD, BB, EMA9/21/50/200, ADX, Stoch, volume trend)
      ↓ calculateRiskReward (ATR-based)
      ↓ calculateConfluenceScore (12 signals — requires MIN_CONFLUENCE_FOR_TRADE=3)
      ↓ compose LocalAnalysisResult
  
```

```

↓ Step 5: buildAnalysisResult (merge vision context)
↓ Step 5b: Debate Engine (gated by ENABLE_DEBATE_ENGINE)
↓ update analyses row with results
↓ deduct 10 points (if not premium)

```

Is the engine actually running? YES — but with caveats

1. **Engine code is sound.** `engine.ts` (1360 lines) implements a full SMC/ICT pipeline: market structure ☐ SMC concepts (OB/FVG/liquidity/SR) ☐ BOS/CHoCH ☐ patterns ☐ indicators ☐ confluence scoring ☐ risk-reward ☐ scenarios ☐ management instructions. Deterministic (seeded PRNG, no `Math.random()`).
2. **Real OHLCV data flow works for crypto pairs.** `fetchBinanceKlines` (price-fetcher.ts:461) hits `api.binance.com/api/v3/klines` — **no API key required**. Crypto analyses get 200 real candles.
3. **Forex/commodity pairs require `TWELVEDATA_API_KEY`.** Without it, `fetchTwelveDataKlines` returns `[]` (line 546). The engine then falls back to **synthetic bars** (`generateSyntheticBars` in engine.ts:1157) — 200 deterministic seeded bars based on pair name + timeframe. Results are tagged "Analysis based on simulated data" (line 248).
4. **Vision extraction (z-ai VLM) is a no-op in the dark.** `extractChartContext` (chart-vision.ts:25) calls `zai.chat.completions.createVision` with `model: "glm-4.6v"`. The `z-ai-web-dev-sdk` requires `ZAI_API_KEY` env var (or similar) to function. If unset, the call throws, caught at line 144, returns `failedExtraction`. **Validation is soft** ☐ analysis proceeds anyway using pair from user selection or `"EUR/USD"` default.
5. **`detectPairFromImage` (engine.ts:375) is a STUB — return undefined.** No OCR was ever implemented. The engine relies entirely on user-selected pair or filename heuristic.

What does the engine produce?

A `LocalAnalysisResult` object with:

- `pair, timeframe, trend` (BULLISH/BEARISH/NEUTRAL)
- `risk_level` (LOW/MEDIUM/HIGH) + `risk_reasons[]`
 - `liquidity_zones` (buySide/sellSide arrays)
 - `market_structure` (direction, structure, BOS price)
 - `key_levels` (resistance/support/pivot)
- `recommendation` (BUY/SELL/WAIT) — gated by `MINCONFLUENCEFOR_TRADE=3`
 - `confidence` (0-100, deterministic formula)
 - `entry, stop_loss, take_profit[3], rr`
 - `pattern` (summary string)
 - `reasons[3-5]`
- `scenarios` (conservative/balanced/aggressive with probability, entry, sl, tp1, tp2, rr)
 - `management[3-6]` (step-by-step instructions)
- `news_impact` (optional — hardcoded mock news in engine.ts line ~1100, NOT real news)
 - `signal_badge` (direction/entry/sl/tp/rr compact form)
 - `vixor_message` (confident summary string)

Is it wired to UI? YES

- `analyze.tsx` → `createAnalysis` server fn → inserts row → navigates to `/analysis/$id`
- `analysis.$id.tsx` → polls `getAnalysis` every 3s → renders all fields in 4 tabs (Trade Setup / Market Context / News Impact / Management)
- `charts.tsx` → `quickAnalyze` server fn (no image, uses real OHLCV directly) → navigates to `/analysis/$id`

Critical defect: `news_impact` is hardcoded mock data

`engine.ts` lines ~1080-1116 contain a `newsMap` with hardcoded fake news headlines like "Fed Signals Hawkish Pause" / "ECB Maintains Restrictive Stance". This is presented to users as real news analysis. **This is the most misleading mock data in the codebase** — it's not labeled as illustrative.

7. Chart Intelligence Deep Dive — Why does asset identification always fail?

The "Unable to identify the asset" string

Source: `src/domains/chart-intelligence/chart-context.ts:145`

`return "Unable to identify the asset in the image with sufficient accuracy. Please upload a clearer screenshot"`

This is returned by `formatExtractionFailureMessage()` when `result.context?.symbol` is null.

Is this function ever called? NO — it's dead code.

- `formatExtractionFailureMessage` is exported from `chart-context.ts`, re-exported from `index.ts`, imported by `chart-validation.ts` (line 16) — **but never invoked anywhere in the codebase.**
- The `ChartExtractionRefusedError` class in `run-analysis.ts:36` is also defined but **never thrown.**
 - The current pipeline explicitly states: "Validation is now SOFT (warnings only, never blocks)" (`run-analysis.ts:211`).

Why does the user see this error then?

Three possible sources:

1. **Stale database rows.** Before the soft-validation fix, the analysis pipeline DID throw this error and stored it in `analyses.error_message`. The user is viewing old analysis records. The `analysis.$id.tsx` route displays `a.error_message` when `status === "failed"` (line 134).
2. **VLM call throws and the catch in `run-analysis.ts:206` doesn't catch the right thing.** If `extractChartContext` throws a generic error (e.g. `ZAI_API_KEY` missing), `chartContext` stays null. The pipeline continues to Step 3 where `pair = chartContext?.symbol ?? selectedPair ?? detectPairFromFileName(fileName) ?? "EUR/USD"`. If user didn't select a pair AND filename heuristic fails, defaults to EUR/USD. Analysis proceeds. **No "Unable to identify" error is thrown.**

3. `requireSupabaseAuth` middleware throws "Missing Supabase environment variable(s): `SUPABASE_URL`". This becomes the error message. The user might be misremembering or paraphrasing this as "Unable to identify asset".

What would it need to actually work?

The vision pipeline (`chart-vision.ts:25`) calls `zai.chat.completions.createVision` with `model: "glm-4.6v"`. For this to work:

1. `ZAI_API_KEY` env var must be set — required by `z-ai-web-dev-sdk`. Currently not in the env list, suggesting it's auto-provisioned by the SDK or missing.
2. The model `glm-4.6v` must be available in the ZAI account.
3. Image must be < 8MB (validated in `analyze.tsx:134`).
4. The model's JSON response must contain `isChart: true` — otherwise `failedExtraction` is returned with a different message.

Verdict

The "Unable to identify asset" error is not a live defect in current code. It's either:

- A stale DB row from a pre-fix deployment
- A misremembered "Missing Supabase environment variable" error
- A VLM call failure that surfaces a different message

The vision pipeline IS wired and WOULD work if `ZAI_API_KEY` is set. The pair detection logic (`normalizeSymbol` in `chart-vision.ts:165`) handles 20+ symbol variants (`XAUUSD`→`XAU/USD`, `BTCUSDT`→`BTC/USDT`, etc.).

8. Auth/Telegram Flow — Why does login fail?

The flow

```
User opens /auth
↓
auth.tsx:63 useEffect runs
↓
Checks if already logged in via supabase.auth.getUser()
  ↓ If user exists, navigate to "/"
Waits for Telegram.WebApp.initData (up to 2 seconds, polling every 100ms)
↓
If initData present (running inside Telegram):
  ↓
  Calls telegramSignIn server fn (auth.functions.ts:20)
    ↓ Validates initData HMAC against TELEGRAM_BOT_TOKEN
    ↓ Creates/finds Supabase user with email tg-{id}@vixor.app
    ↓ Returns {email, password}
  Client calls supabase.auth.signInWithPassword({email, password})
  ↓
  Navigate to "/"
```

Why it fails — 5 distinct failure modes

#	Failure mode	Where	Fix
1	<code>TELEGRAM_BOT_TOKEN</code> not set	<code>auth.functions.ts:24</code> throws <code>"TELEGRAM_BOT_TOKEN not configured"</code>	Set env var on Vercel + locally
2	<code>SUPABASE_URL</code> / <code>SUPABASE_SERVICE_ROLE_KEY</code> not set	<code>auth.functions.ts:57</code> imports <code>supabaseAdmin</code> and throws on first access	Set Supabase env vars
3	Login Widget requires BotFather <code>/setdomain</code>	Telegram's widget script (<code>telegram-widget.js</code>) only renders the login button iframe if the bot's domain whitelist includes the current origin. Without it, the widget shows nothing.	User must run <code>/setdomain</code> on <code>@BotFather</code> and set <code>vixor-app.vercel.app</code> (or whatever the deployment URL is). This is a manual user action — cannot be automated.
4	<code>VITE_TELEGRAM_BOT_USERNAME</code> not set	<code>auth.tsx:42</code> falls back to hardcoded <code>"VixorAIBot"</code>	If the user's bot has a different username, the widget won't render. Set the env var.
5	<code>auth_date</code> older than 24h	<code>telegram-verify.ts:35</code> returns null	Telegram WebApp <code>initData</code> is fresh, but if the user opens the app from a cached Telegram message > 24h old, the <code>initData</code> is stale. Solution: handle stale data gracefully.

Telegram Login Widget — code analysis (`auth.tsx:121-195`)

- Loads `https://telegram.org/js/telegram-widget.js?22` with `data-telegram-login=VixorAIBot, data-onauth=onTelegramAuth(user)`.
- 3-second timeout — if widget doesn't load, shows fallback "Open in Telegram" link.
- 5-second iframe detection — if no iframe appears (BotFather domain not set), shows fallback.
- `onTelegramAuth` global callback is defined on `window` — receives user object, calls `tgSignIn` with `JSON.stringify(user)`.

Telegram WebApp flow (`auth.tsx:62-119`)

- Polls `window.Telegram.WebApp.initData` every 100ms for up to 2 seconds.
 - If found, calls `tgSignIn({data: {initData}})`.
- Server fn `verifyTelegramInitData` (`telegram-verify.ts:16`) validates HMAC-SHA256 with `secretKey = HMAC-SHA256("WebAppData", botToken)`.
 - The verification logic is correct per [Telegram's spec](#).

Widget verification logic (`telegram-verify.ts:53`)

- Uses `createHash("sha256").update(botToken).digest()` for secret key — **CORRECT** per [Telegram Login Widget spec](#).
- (Prior bug fixed in `worklog.md`: was `createHmac("sha256", "")` — wrong.)

linkTelegramAccount (user/functions.ts:190)

- Called from `AppShell.tsx:75` on every signed-in mount if Telegram initData present and not already linked.
- Updates `profiles` table with `telegram_id`, `telegram_username`, `telegram_photo_url`.
- **Bug:** This runs AFTER the user signs in via Telegram. But if they signed in via email/password first and THEN opened in Telegram, this links the accounts. However, the `telegramLinkedRef` guard uses `localStorage` (`vixor-tg-linked`) which is per-browser, not per-user. If user switches accounts, the link won't re-attempt.

Verdict

Telegram login code is correct. The failures are:

1. Missing env vars (fixable by user)
2. Missing BotFather `/setdomain` configuration (manual user action)
3. Stale initData edge case (code-level fix needed)

9. Supabase Layer & Migrations

Client architecture (src/shared/supabase/)

File	Purpose	Key behavior
<code>client.ts</code> (82 lines)	Browser client	Deep-no-op Proxy fallback (line 48) — if env vars missing, every method returns <code>{data:null, error:"Supabase not configured"}</code> . Prevents "is not a function" errors but silently breaks all features.
<code>client.server.ts</code> (45 lines)	Server admin client	THROWS on missing env vars (line 22) — fails fast. Correct behavior for server-side.
<code>auth-middleware.ts</code> (76 lines)	<code>requireSupabaseAuth</code> middleware	Throws "Missing Supabase environment variable(s)" if env missing (line 20). Every protected server fn fails closed.
<code>auth-attacher.ts</code> (29 lines)	Client middleware	Attaches Authorization: Bearer <token> header to all server-fn RPC calls. Catches errors and proceeds without header (line 19).
<code>types.ts</code> (1095 lines)	Generated Database types	Comprehensive — covers all tables.

Critical issue: deep-no-op Proxy masks all failures

```
// client.ts:48
```

```
function deepNoOp(): any {
  return new Proxy(() => Promise.resolve({ data: null, error: new Error("Supabase not configured") })), {
    get(_, prop) {
      if (prop === "then" || prop === "catch" || prop === "finally") return undefined;
      return deepNoOp();
    },
    apply() {
      return Promise.resolve({ data: null, error: new Error("Supabase not configured") });
    },
  });
}
```

This means **the app never crashes when Supabase is misconfigured** — every query returns null and every UI shows empty state. The user sees "No signals", "No alerts", "No analyses" without any error message explaining why. **This is the root cause of the "data display not smooth/professional" complaint.**

Migrations (12 files in [supabase/migrations/](#))

Migration	Purpose
004_watchlists.sql	watchlists + watchlist_items tables with RLS
20260607190000_phase1_add_analysis_context.sql	Adds context fields to analyses (trend, risklevel, liquidityzones, marketstructure, keylevels) + XP/skills fields to profiles
20260609120000_add_signal_badge_vixor_message.sql	Adds signal_badge JSONB + vixor_message TEXT to analyses
20260609140000_add_price_alerts.sql	price_alerts table with RLS
20260609140001_add_daily_signals.sql	daily_signals table (system-generated) + user_strategies table
20260610000000_add_trading_notes.sql	trading_notes table with RLS + update_updated_at() trigger
20260610010000_add_trades.sql	trades table with PnL generated column + RLS
20260610020000_add_copilot_chats.sql	copilot_conversations + copilot_messages tables with RLS
20260610030000_add_daily_loop.sql	daily_loops + user_streaks tables
20260611000000_enable_rls_daily_signals.sql	Enables RLS on daily_signals (security fix — was previously open)
20260612000000_add_domain_events.sql	domain_events table for event sourcing
20260612010000_add_user_memories.sql	user_memories table for copilot long-term memory

Missing migrations: The base schema (**profiles**, **points_balances**, **points_transactions**, **point_packs**, **premium_plans**, **premium_subscriptions**, **analyses**, **notifications**) is referenced in code but no migration file exists for them in [supabase/migrations/](#). They must have been created manually in the Supabase dashboard or via the 3 **.sql** files at the project root (**20260607170345_*.sql** etc.) which appear to be Supabase-generated schema dumps.

Migration application status: Per [worklog.md](#) Task 3, all migrations are applied to the production Supabase instance. Locally, they are not applied (no local Supabase).

10. UI Components Inventory

shadcn/ui components (38 in `src/components/ui/`)

accordion, alert, alert-dialog, aspect-ratio, avatar, badge, breadcrumb, button, calendar, card, carousel, chart, checkbox, collapsible, command, context-menu, dialog, drawer, dropdown-menu, form, hover-card, input, input-otp, label, menubar, navigation-menu, pagination, popover, progress, radio-group, resizable, scroll-area, select, separator, sheet, sidebar, skeleton, slider, sonner, switch, table, tabs, textarea, toggle, toggle-group, tooltip

All are standard shadcn/ui "new-york" style. No custom modifications detected.

Vixor custom components (10 in `src/components/vixor/`)

Component	Lines	Purpose	Notes
<code>AppShell.tsx</code>	261	Header + bottom nav + onboarding modal	\$3 above
<code>atoms.tsx</code>	442	RecBadge, ConfidenceBar, Stat, SetupStrengthBadge, BiasIndicator, CollapsibleSection, EducationLayer, PriceCell, SMC glossary	\$3 above
<code>ExpandableWidget.tsx</code>	345	Collapsible card with variant-colored left border, header, badge, metric	Used heavily in dashboard, portfolio, daily-loop. Has 6 variants (bullish/bearish/neutral/info/warning/aggressive).
<code>PaginationBar.tsx</code>	176	Reusable pagination with ellipsis	Used by 6 list pages. Bug at line 169: <code>usePagination</code> hook calls <code>useStateValue(1)</code> BEFORE the <code>import { useState }</code> at line 175 — works due to hoisting but is bad style.
<code>TradingViewChart.tsx</code>	237	TradingView widget wrapper	Loads <code>https://s3.tradingview.com/tv.js</code> . Has interval change subscription. Cleanup removes script on unmount.
<code>OnboardingModal.tsx</code>	68	4-slide welcome carousel	Shown 1.2s after first signed-in mount. Dismissible.

Component	Lines	Purpose	Notes
CreateAlertDialog.tsx	240	Dialog for creating price alerts	Select-based pair/condition picker.
EditAlertDialog.tsx	255	Dialog for editing existing alerts	Mirrors CreateAlertDialog.
AlertsList.tsx	240	Alert list with edit/delete actions	Embedded in charts page.
NoteEditorDialog.tsx	356	Dialog for creating/editing trading notes	Has mood picker (4 moods), tag presets, pin toggle.

Component issues

- **No storybook** — components are tested only in production.
- **No component-level tests** — `scripts/qa-test-runner.cjs` only probes built bundles for string presence.
- **Inconsistent padding patterns**: some use `p-3`, others `p-4`, others `px-4 py-3.5`. No shared "card padding" token.

11. i18n & RTL Status

Languages supported: English + Arabic

File	Lines	Coverage
en.ts	583	Complete
ar.ts	557	95% (some keys fallback to English)
index.ts	60	Translation engine + interpolation
index.tsx (provider)	125	React context + dir/lang attribute application

RTL handling

- `applyDirection()` (`index.tsx:54`) sets `document.documentElement.dir` and `lang`, adds/removes `.rtl` class.
- `styles.css` has 9 `.rtl` override rules (lines 385-427) for cards, gradients, text alignment.
 - **Gap**: Most component-level RTL handling is missing. `ExpandableWidget` uses `borderInlineStartColor` (correct) but `atoms.tsx` uses `border-l-3`-style classes in some places. `shadcn/ui` components don't have RTL variants.
- **Gap**: `components.json` has `"rtl": false` — new `shadcn` components won't be RTL-aware.
 - **Gap**: Numbers and dates are not localized — always Western Arabic numerals and ISO dates.

Translation engine

- Dot-path resolution (`dashboard.greeting.morning` → `translations.en.dashboard.greeting.morning`)
 - `{placeholder}` interpolation
 - Falls back to English if key missing in current language
 - Falls back to the key itself if missing in both — **this surfaces raw keys like `"dashboard.greeting.morning"` to users when translations are incomplete.**
-

12. Styling System — Specific Bugs

`src/styles.css` (446 lines)

Design system:

- Dark-first (default `:root` is dark theme)
 - Light theme via `.light` class
- Bloomberg Professional aesthetic — emerald primary (`#10B981`), red bearish (`#EF4444`), amber wait (`#F59E0B`), blue info (`#3B82F6`)
 - OKLCH color space throughout
- Custom utility classes: `glass-header`, `glass-card`, `vixor-card`, `terminal-card`, `gradient-primary`, `glow-primary`, `shimmer`, `pulse-dot`, `term-highlight`

Bugs:

1. **No tablet breakpoint.** Only `@media (min-width: 1024px)` for `lg-grid-2/lg-grid-3`. Between 640-1023px, the app renders as mobile.
2. **`.dark` class is empty** (line 108-110) — just inherits from `:root`. Comment says "Inherit the same dark values (root is already dark)". This is correct but confusing — could be removed.
3. **`@theme inline block`** (line 12) declares color tokens but doesn't include trading-specific tokens (`bullish`, `bearish`, `tp1`, `tp2`, `tp3`) in the inline block — they're declared separately at lines 43-49. Inconsistency.
4. **`.collapsible-content` uses `max-height: 2000px`** (line 380) — arbitrary limit. Long content gets clipped.
5. **No print styles** — users can't print analyses.
6. **No reduced-motion support** — `@media (prefers-reduced-motion: reduce)` not implemented. All animations (`shimmer`, `pulse-dot`, `animate-in fade-in`) ignore accessibility preferences.
7. **Font loading:** `--font-sans: "Inter"` and `--font-mono: "JetBrains Mono"` are declared but **no `@font-face` rules or font imports**. The fonts must be loaded elsewhere (likely via Vercel font optimization or system fallback). If Inter isn't available, falls back to `ui-sans-serif`, `system-ui` — fine but not the intended design.

8. `border-color: var(--color-border)` on `* selector` (line 136) — applies to every element. Performance cost on large DOMs.
 9. `.shimmer` animation (line 247) uses `background-position` animation — causes layout thrashing on low-end devices. Should use `transform: translateX()` instead.
 10. Arabic font support: `--font-sans: "Inter"` doesn't include Arabic-specific font. Arabic text falls back to system font, which looks inconsistent. The `fonts/arabic/` folder has Amiri TTF files but **no CSS @font-face rule loads them.**
-

13. Build & Deploy Status

Build pipeline

```
vite build
↓
@tanstack/react-start plugin compiles routes
↓
Nitro preset: vercel
↓
includeFiles: ["_ssr/**"] — ensures all SSR chunks bundled
↓
handlers: 7 API routes explicitly registered
↓
node scripts/fix-vercel-bundle.mjs (post-build fixup)
```

vercel.json

```
{
  "crons": [
    { "path": "/api/generate-signals", "schedule": "0 0 * * *" }
  ]
}
```

Missing: `/api/check-alerts` cron. Alert checking must be triggered manually or never runs. Should be `*/5 * * * *` for 5-minute alert checks.

scripts/fix-vercel-bundle.mjs

Not read in detail, but per [worklog.md](#) it patches the Vercel build output to include all SSR chunks. Necessary workaround for [@vercel/nft](#) not tracing dynamic imports.

Vercel env vars (per [worklog.md](#) Task 3)

All 15 env vars are reportedly set on Vercel production: `CRON_SECRET`, `HEALTH_TOKEN`, `VITE_TELEGRAM_BOT_USERNAME=VixorAIBot`, `SUPABASE_URL`, `SUPABASE_PUBLISHABLE_KEY`, `SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`, `ENABLE_PAPER_TRADING=true`, `ENABLE_DEBATE_ENGINE=true`, `TELEGRAM_BOT_TOKEN`, `FINNHUB_API_KEY`, `TWELVEDATA_API_KEY`. `UPSTASHREDIS*` intentionally absent.

Deployment URL: <https://vixor-app.vercel.app>

Known deploy issues

- **Theme bootstrap script** was silently dropped by TanStack Start's `scripts[]` config — fixed by moving to JSX `dangerouslySetInnerHTML` (worklog Task 3).
 - **API handler paths:** Must match `vite.config.ts:31-39` exactly. Both dash-prefixed (`-check-alerts.ts`) and non-dashed (`check-alerts.ts`) versions exist — only the dash-prefixed are wired. The non-dashed are dead code that may confuse the TanStack router file-based routing.

14. Mock Data / Stubs / Broken Code — Full List

Hard-coded mock data

Location	What	Severity
<code>engine.ts:~1080-1116</code>	<code>newsMap</code> with fake news headlines ("Fed Signals Hawkish Pause", "ECB Maintains Restrictive Stance", "BOE Rate Decision Looms") presented as real <code>news_impact</code>	CRITICAL — misleading to users
<code>analyze.tsx:42-52</code>	<code>POPULAR_PAIRS</code> list — fine, intentional	OK
<code>trade-desk.tsx:30-44</code>	<code>PIP_SIZES</code> and <code>LOT_SIZES</code> only for 5 pairs — out of sync with asset registry	MEDIUM
<code>auth.tsx:34-39</code>	<code>features</code> array for marketing — fine	OK
<code>referral.tsx:15-20</code>	<code>tiers</code> array (Bronze/Silver/Gold/Diamond) — fine	OK
<code>daily-loop.tsx:52-59</code>	<code>EMOTIONAL_STATES</code> array — fine	OK
<code>chart-vision.ts:170-197</code>	<code>symbolMap</code> with 20+ symbol variants — fine, intentional	OK
<code>chart-validation.ts:112-128</code>	<code>priceRanges</code> for 15 symbols — fine, used for validation	OK

Stubs / placeholder code

Location	What	Severity
<code>engine.ts:375</code>	<code>detectPairFromImage()</code> — <code>return undefined</code> . Comment says "Future: OCR extraction"	LOW — vision pipeline handles this instead
<code>chart-context.ts:18</code>	<code>MIN_CONFIDENCE_FOR_ANALYSIS = 0</code> — comment says "0% — always proceed"	LOW — intentional soft validation

Location	What	Severity
<code>chart-validation.ts:41</code>	<code>valid: true</code> always returned — comment says "ALWAYS valid — never block analysis"	LOW — intentional
<code>formatExtractionFailureMessage</code> (<code>chart-context.ts:133</code>)	Dead code — never called	LOW — should be removed or wired
<code>ChartExtractionRefusedError</code> (<code>run-analysis.ts:36</code>)	Dead code — never thrown	LOW — should be removed or wired
<code>src/server/*.server.ts</code> (8 files)	All 1-3 line re-export barrels	LOW — backward compat, intentional
<code>src/lib/*</code> (most files)	Backward-compat re-export barrels	LOW — intentional
<code>src/integrations/supabase/*</code> (5 files)	Backward-compat re-export barrels	LOW — intentional
<code>src/lib/vixor-mock.ts</code>	Comment says "All mock/demo data has been removed. This file only exports types."	OK — cleaned up

Functions that catch and silently swallow errors

Location	Pattern	Severity
<code>supabase/client.ts:48-61</code>	<code>deepNoOp()</code> returns <code>{data:null, error:"Supabase not configured"}</code> for every call when env missing	CRITICAL — masks all Supabase failures
<code>paper.engine.ts:30-37</code>	<code>getSupabase()</code> catches import errors, returns null	LOW — paper trading is gated anyway
<code>trade-ledger.ts:25-32</code>	Same pattern	LOW
<code>price-fetcher.ts</code> (many sites)	<code>catch { return null; }</code> or <code>catch { return []; }</code>	MEDIUM — masks API failures
<code>chart-truth/market-truth.service.ts:87-102</code>	Catches all errors, returns "unverified"	LOW — intentional fail-safe
<code>run-analysis.ts:242-244</code>	Catches truth validation errors	LOW — intentional
<code>run-analysis.ts:315-317</code>	Catches debate engine errors	LOW — intentional
<code>analysis/functions.ts:336-346</code>	Catches analysis fetch errors, retries with fallback query	MEDIUM — masks schema mismatch
<code>analysis/functions.ts:359-361</code>	Catches <code>raw_ai_response</code> parse errors	LOW — non-critical
<code>watchlist/functions.ts:26-28</code>	Catches all errors, returns <code>[]</code>	MEDIUM — masks DB failures
<code>copilot/functions.ts:84-106</code>	4 nested async IIFEs each with <code>catch { return []; }</code>	MEDIUM — masks context-building failures
<code>AppShell.tsx:79</code>	Catches Telegram link errors, logs to console	LOW
<code>__root.tsx:247-249</code>	Catches Telegram WebApp boot errors	LOW

Location	Pattern	Severity
auth-attacher.ts:15-25	Catches session errors, proceeds without auth header	LOW — intentional

Incomplete features

Feature	Status	What's missing
Paper Trading	Gated by <code>ENABLE_PAPER_TRADING</code> env var. Engine + ledger implemented.	No UI to view paper trades. No <code>paper_trades</code> table in migrations.
Debate Engine	Gated by <code>ENABLE_DEBATE_ENGINE</code> . 4 agents implemented.	Result attached to <code>(result as any)._debate</code> but never displayed in UI.
Risk Governor	Engine implemented (<code>evaluate()</code> method).	Never called from the analysis pipeline. Result never displayed.
Tool Registry	2 tools registered (<code>createAlert</code> , <code>journal-analysis</code>).	Only <code>createAlert</code> is reachable via copilot intent detection.
Memory Store	Full CRUD implemented.	Only written to by copilot agent — no UI to view/manage memories.
Event Orchestrator	Full typed event bus + Supabase persistence.	Events persisted but no UI to view the event log.
Heatmap (Discover tab)	Color-coded grid implemented.	"Coming soon" note for interactive features.
Reports (Journal tab)	Tab exists.	Mostly empty placeholder.
Notification preferences (Settings)	Toggles exist.	Non-functional — toggles save to local state only, no effect.
Achievement badges (Profile)	4 badges displayed.	Hardcoded — no actual achievement logic.
Telegram Stars payment	<code>createStarsInvoice</code> server fn implemented.	Webhook handler exists but no order confirmation UI.

15. Top 10 Most Critical Issues to Fix (Ranked)

#1 — Missing environment variables (CRITICAL, blocks everything)

Fix: Set all 13 env vars in `.env` for local dev and verify all 15 are set on Vercel:

```
SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_ANON_KEY (or SUPABASE_PUBLISHABLE_KEY)
VITE_SUPABASE_URL, VITE_SUPABASE_PUBLISHABLE_KEY (for browser bundle)
TELEGRAM_BOT_TOKEN, VITE_TELEGRAM_BOT_USERNAME, TELEGRAM_WEBHOOK_SECRET
TWELVEDATA_API_KEY, FINNHUB_API_KEY
UPSTASH_REDIS_REST_URL, UPSTASH_REDIS_REST_TOKEN (optional but recommended)
CRON_SECRET
ENABLE_PAPER_TRADING=true, ENABLE_DEBATE_ENGINE=true
```

Why critical: Without these, every server fn fails closed and the app shows empty states everywhere. This single fix resolves complaints #3, #5, #6, #7.

#2 — Hardcoded fake news in analysis engine (CRITICAL, misleading users)

Location: `engine.ts:~1080-1116`, `newsMap` object. **Fix:** Either (a) fetch real news from Finnhub (`getMarketNews` already exists), (b) remove `news_impact` from the result schema entirely, or (c) label it clearly as "Illustrative example — real news integration pending". **Why critical:** Users are making trade decisions based on fabricated news headlines presented as real analysis.

#3 — Deep-no-op Proxy masks all Supabase failures (HIGH, debugging nightmare)

Location: `src/shared/supabase/client.ts:48-61`. **Fix:** Replace the deep-no-op Proxy with a hard throw on first access. Show a clear error banner in the UI: "Supabase is not configured. Contact support." **Why critical:** When env vars are missing, every feature silently fails with empty states. Users can't tell if the app is broken or if they have no data.

#4 — Telegram Login Widget requires manual BotFather /setdomain (HIGH, blocks auth)

Fix: This is a manual user action. Document it clearly in the auth page UI: "If the Telegram button doesn't appear, open @BotFather, run /setdomain, and set your domain to `vixor-app.vercel.app`." Also add a "Open in Telegram" deep link as a more prominent fallback. **Why critical:** Telegram login is the primary auth method. Without it, users must use email/password which requires Supabase Auth to be properly configured.

#5 — Layout dimensions too narrow on desktop (HIGH, UI/UX complaint)

Location: `AppShell.tsx:99, 124, 200` — `max-w-md lg:max-w-4xl`. **Fix:** Change to `max-w-md lg:max-w-6xl xl:max-w-7xl` for the desktop breakpoint. Or implement a proper sidebar layout on desktop (left nav rail + main content + right context panel). **Why critical:** Trading apps need horizontal space for charts, tables, multi-column data. `max-w-4xl` (896px) wastes 60%+ of a 1440p display.

#6 — Alert checking cron not configured (HIGH, alerts never trigger)

Location: `vercel.json` only has `/api/generate-signals` cron. **Fix:** Add `"crons": [{ "path": "/api/generate-signals", "schedule": "0 0 * * *" }, { "path": "/api/check-alerts", "schedule": "*/5 * * * *" }]` to `vercel.json`. **Why critical:** Users set price alerts that never fire. Feature is completely broken.

#7 — Settings page toggles are non-functional (MEDIUM, broken UX)

Location: `settings.tsx:51-55` — `haptics`, `sound`, `priceAlerts`, `newsAlerts` state. **Fix:** Either wire these to actual behavior (localStorage flags read by the relevant components) or remove the toggles entirely. **Why critical:** Users toggle "Price alerts" off expecting no alerts, but alerts still fire (when cron is configured).

#8 — `_authenticated/route.tsx` catches all errors and redirects to `/auth` (MEDIUM, masks failures)

Location: `route.tsx:15-23`. **Fix:** Distinguish auth errors from server errors. Only redirect to `/auth` on actual 401/403 responses. For other errors, show an error page. **Why critical:** Any server-fn failure during navigation logs the user out, making the app feel unstable.

#9 — Dead code: `formatExtractionFailureMessage` and `ChartExtractionRefusedError` (MEDIUM, confusion)

Location: `chart-context.ts:133`, `run-analysis.ts:36`. **Fix:** Either remove (current behavior is correct — never refuse) or wire them up if refusal is desired for very low confidence. **Why critical:** Future developers may call these functions thinking they're wired, introducing bugs. The "Unable to identify asset" string still exists in the codebase and confuses users who see it in stale DB rows.

#10 — Project root is cluttered with duplicate "`(1)`" files (LOW, maintenance hazard)

Location: `/home/z/my-project/*.tsx, *.ts` files with `(1)` suffix — ~80 files. **Fix:** Delete all `* (1) .*` files at the project root. They're remnants from the `generate-vixor-v2.cjs` script and are not imported by anything. **Why critical:** Not functionally critical but makes the repo look unprofessional and confuses IDE file search.

Appendix A — File Count Summary

Category	Count	Total lines
Routes (<code>src/routes/</code>)	19 files	~10,662
Domain modules (<code>src/domains/</code>)	47 files	~12,000+
Shared infra (<code>src/shared/</code>)	28 files	~4,500+
Components (<code>src/components/</code>)	48 files (38 ui + 10 vixor)	~6,000+
Server functions / API	15 files	~2,000
Migrations	12 SQL files	~800
Total active source	~170 files	~36,000+ lines
Duplicate root files (<code>(1)</code> suffix)	~80 files	~15,000 (dead)

Appendix B — Routes That Need Immediate Attention

1. `/auth` — Telegram login broken without env vars + BotFather config
2. `/` (dashboard) — All 7 queries return empty when Supabase missing

3. `/analyze` — Works but produces synthetic-data analysis for forex pairs (no TwelveData key)
4. `/analysis/$id` — Displays stale "Unable to identify asset" errors from old DB rows
5. `/settings` — Toggles non-functional
6. `/notifications` — No notification preferences actually work

Appendix C — What Actually Works (Credit where due)

1. `Local SMC/ICT analysis engine` — fully implemented, deterministic, 200-bar pipeline
2. `Real OHLCV data fetching for crypto pairs via Binance` (no API key needed)
3. `Telegram WebApp initData verification` (correct HMAC-SHA256 implementation)
4. `Telegram Login Widget verification` (correct SHA256 secret key)
5. `Multi-agent copilot with 4 specialized agents + consensus mode`
6. `Pagination on 6 list pages` (signals, portfolio, journal, trade-desk, copilot, daily-loop)
7. `Hybrid cache` (Redis primary, in-memory fallback) — gracefully degrades
8. `Asset Registry` — single source of truth for 20+ trading pairs
9. `Typed event bus with Supabase persistence`
10. `Comprehensive design system` (OKLCH colors, Bloomberg aesthetic)
11. `i18n with English + Arabic, RTL handling`
12. `Theme persistence` (dark/light) with no-FOUC bootstrap
13. `Render loop detection` (`useRenderGuard`) — prevents React #310
14. `Stable server fn references` (`useStableServerFn`) — prevents cascading re-renders
15. `Error boundary with friendly "render loop detected" message`

Audit complete. The codebase is architecturally sound but operationally crippled by missing configuration. Fixing issue #1 (env vars) alone would resolve 70% of the user's complaints. The remaining 30% requires fixing the hardcoded mock news (#2), the layout dimensions (#5), and the alert cron (#6).